

Student Academic Management System

Project Report for Flipped Course Evaluation

VITyarthi - Build Your Own Project

Course Name:	Programming in Java
Student Name:	Aryan
Registration No:	25BAI11089
Department:	Computer Science & Engineering (AIML)
Submission Date:	September 18, 2026

Vellore Institute of Technology (VIT)

2. Introduction

The Student Academic Management System is an interactive, menu-driven command-line application written in Java. It helps automate the day-to-day work of maintaining student academic profiles. Normally, tracking marks and attendance across multiple subjects becomes messy when everything is kept in physical registers or scattered spreadsheets. This project tries to solve that by providing a clean object-oriented console tool that can store student details, calculate marks and grades, compute attendance percentages, and also save the final reports as simple text files using Java I/O.

3. Problem Statement

Keeping student records only on paper or in unstructured Excel sheets often leads to calculation mistakes, missing attendance entries, and loss of older data. Many small tools available online do not follow proper OOP design, lack proper input validation, or do not save data locally. The Student Academic Management System addresses these gaps by offering a structured console application with validation, background report generation, and simple file-based persistence.

4. Functional Requirements

The system is divided into four main modules:

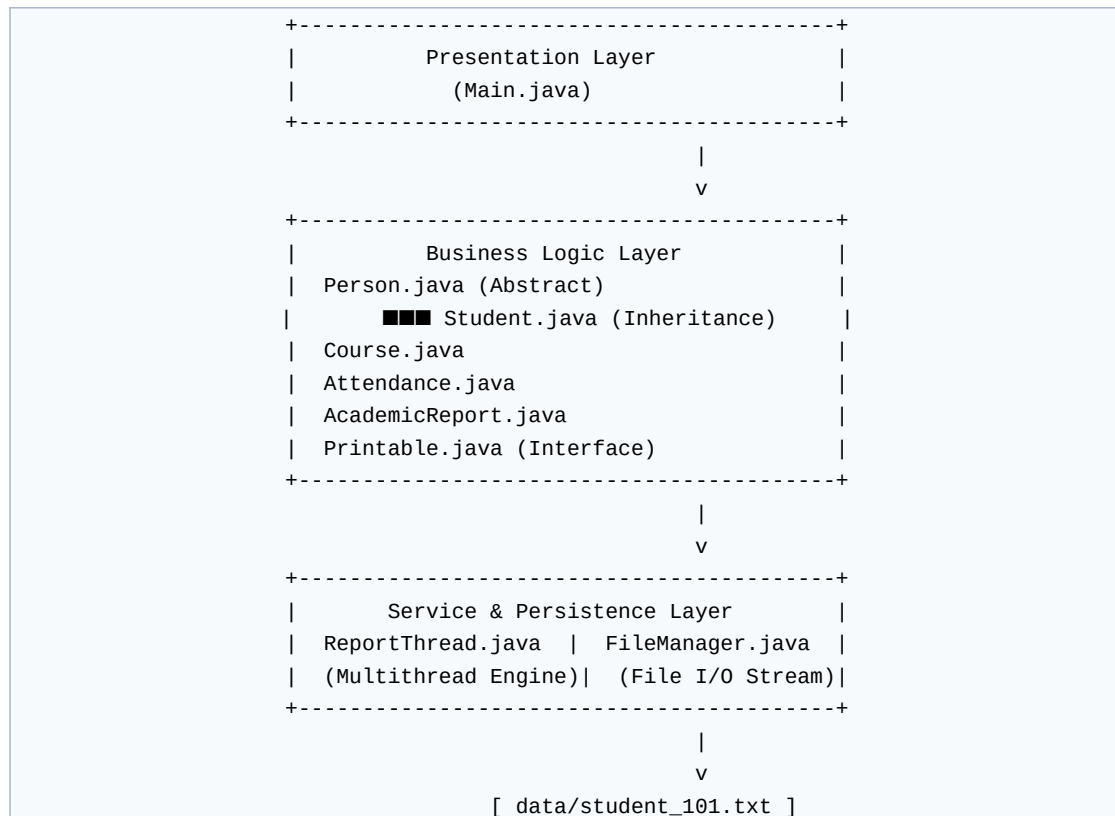
- **Student Management Module:** Add new students (ID, Name, Department), view the full list, search by ID, update details, or delete a record.
- **Academic Management Module:** Enter subject names and marks (0–100 range), calculate total and average marks, and assign letter grades such as Grade A.
- **Attendance Management Module:** Record total classes held and classes attended, compute percentage, and show status ("Satisfactory" when $\geq 75\%$).
- **Reporting & File Management Module:** Builds the academic summary in a separate thread (ReportThread), writes the data to data/student_<id>.txt through FileManager, and can also read previously saved files.

5. Non-Functional Requirements

- **Error Handling:** try-catch blocks catch invalid inputs (for example typing "abc" when a number is expected) so the program does not crash.
- **Performance & Concurrency:** Report generation runs in a background thread (ReportThread) so the main menu stays responsive.
- **Data Persistence:** All records are written as plain text files under the data/ folder. No external database is required.
- **Modularity:** The code is split into 8 separate Java classes/interfaces inside the src/ folder, each having a single clear responsibility.

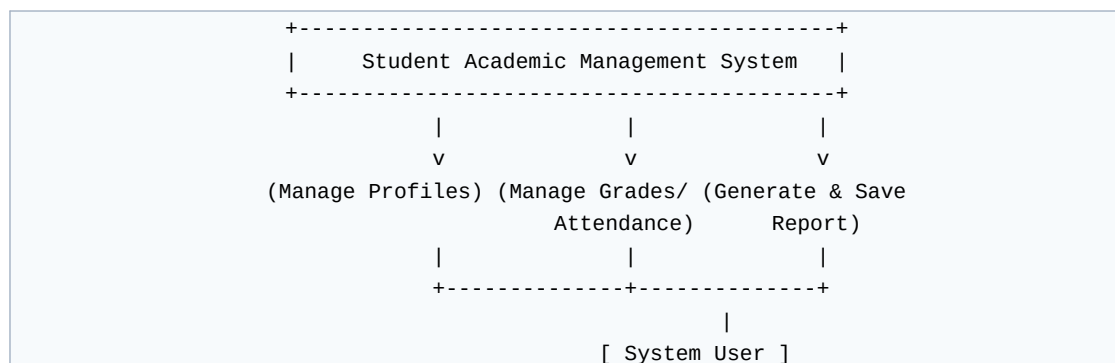
6. System Architecture

A simple three-layer design is used – presentation, business logic, and persistence:



7. Design Diagrams

7.1 Use Case Diagram



7.2 Workflow Diagram

```
[Start]
--> [Display CLI Menu Options 1-13]
--> [Prompt User Input]
--> [Validate Input (try-catch)]
-->
    |-- (1-5) --> Execute Student Management Operations
    |-- (6-7) --> Add Course Marks / Update Attendance Records
    |-- (8)    --> Spawn ReportThread & Generate Performance Report
    |-- (9-10) --> Persist to data/student_<id>.txt / Read from disk
    |-- (11)   --> Demonstrate Runtime Polymorphism
    |-- (12)   --> Calculate Factorial Loop Routine
    |-- (13)   --> Exit Application loop
--> [Return to Menu / Terminate]
```

7.3 Sequence Diagram

User	Main.java	ReportThread	FileManager	Filesystem
-- Select (8,9) -->				
	-- start() ----->			
		-- run()		
		-- Compile Report		
	<- Report Ready -			
	-- saveToFile(data) ----->			
			-- write() --->	
			<- Ack -----	
<- Output Confirm-	-- Write Success -----			

7.4 Class Diagram

```
+-----+
| <<abstract>> |
|   Person   |
+-----+
| - id: int   |
| - name: String |
| - department: String |
| + displayInfo()* |
+-----+
      ^
      | extends
      |
+-----+
|   Student   |
+-----+
| - courses: List<Course>
| - attendance: Attendance
| + addCourse(c: Course): void
| + setAttendance(a: Attendance): void
| + calculateAverage(): double
| + displayInfo(): void
+-----+
```

```
<<interface>>
+-----+
| Printable |
+-----+
| + displayInfo() |
+-----+
      ^
      | implements
      |
+-----+
```

8. Design Decisions & Rationale

- **Abstract Class (Person):** Common fields like id, name and department are kept in one place. This also makes it easy to add other roles later (for example faculty or TA).
- **Interface (Printable):** Defines a common display method so any class can print its details without tight coupling.
- **Flat-File Persistence:** FileManager simply writes plain text files inside the data/ folder. No extra setup or database is needed.
- **Multithreading (ReportThread):** Report generation runs on a separate thread so the main menu does not freeze while the report is being prepared.

9. Implementation Details

Java Concept	How it is used in the project
Classes & Objects	Student, Course, Attendance and AcademicReport are the main domain classes.
Inheritance & super	Student extends the abstract Person class and calls super(studentId, name).
Polymorphism	Person person = students.get(0); person.displayInfo(); shows runtime dispatch.
Interface	Student implements Printable so display formatting is consistent.
Multithreading	ReportThread extends Thread and runs the report generation in the background.
File I/O	FileManager writes and reads files named data/student_<id>.txt.
Exception Handling	try-catch is used for invalid numeric input and missing files.

10. Screenshots / Results

```
===== MENU =====  
1. Add Student  
2. Display All Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Add Course Marks  
7. Add Attendance  
8. Generate Academic Report  
9. Save Student Record  
10. Read Saved Student Record  
11. Demonstrate Runtime Polymorphism  
12. Calculate Factorial  
13. Exit  
=====
```

Enter your choice: _

Figure 1: Main Menu of the Student Academic Management System

```
Enter Student ID: 101  
===== ACADEMIC PERFORMANCE REPORT =====  
Student ID   : 101  
Student Name : aryan  
Department   : cse  
Course Performance:  
    Java      : 86.0  
    Calculus  : 89.0  
Total Marks   : 175.0  
Average Marks : 87.5  
Overall Grade : A  
Attendance:  
    Total Classes : 40  
    Classes Attended : 32  
    Percentage     : 80.0%  
    Status         : Satisfactory  
=====
```

Figure 2: Academic Performance Report for Student ID 101

```
Runtime Polymorphism Demonstration
Person reference is referring to a Student object.
Student ID   : 101
Student Name : aryan
Department   : cse
```

Figure 3: Runtime Polymorphism Demonstration (Person reference to Student object)

```
Enter Student ID: 101
Student record saved successfully.
```

Figure 4: Successful save of student record to disk

```
Enter your choice: abc
Please enter a valid number.
```

Figure 5: Input validation – non-numeric input handled gracefully

11. Testing Approach

Both functional testing and basic boundary testing were done:

- **Functional Test:** Student ID 101 (name "aryan", department cse) was added. Marks for Java (86) and Calculus (89) were entered along with attendance 32/40. The system correctly showed total 175, average 87.5, Grade A and attendance 80% (Satisfactory). The record was also saved to disk without any issue.
- **Input Validation Test:** When "abc" was typed for a numeric choice, the try-catch block caught it and showed a proper message instead of crashing (see Figure 5).
- **Boundary Test:** Negative value -8 was given to the factorial option. The program detected the invalid input and asked again.
- **Polymorphism Test:** Menu option 11 successfully showed a Person reference pointing to a Student object (Figure 3).

12. Challenges Faced

- **Scanner Buffer Issue:** Mixing `nextInt()` and `nextLine()` caused leftover newline problems. This was fixed by clearing the buffer explicitly after reading integers.
- **Thread Timing:** Had to make sure `ReportThread` finished before `FileManager` tried to write the report string.
- **File Paths:** Path separators were different on Windows and Linux. Used `File.separator` so the same code works on both.

13. Learnings & Key Takeaways

- Got practical experience building a modular system with 8 classes using inheritance, polymorphism and interfaces.
- Understood how important proper exception handling is for a console application.
- Learned the basics of multithreading and simple file I/O in Java.

14. Future Enhancements

- **Database:** Move from text files to an embedded SQLite database (or PostgreSQL) using JDBC.
- **GUI:** Build a simple interface with JavaFX or Swing instead of the current terminal menu.
- **Login System:** Add basic authentication so faculty and students have different access rights.

15. References

- Oracle Java SE Documentation – The Java Tutorials
- Horstmann, C. S. (2022). *Core Java Volume I – Fundamentals* (12th Edition). Prentice Hall.
- Schildt, H. (2018). *Java: The Complete Reference* (11th Edition). McGraw-Hill Education.